Figure 10.1 You can store the default map location, zoom level, and mapping API Key in configuration and load them at runtime. It's important to keep secrets such as API keys in configuration out of your code.

There's also a security aspect: you don't want to hardcode secret values such as API keys or passwords into your code, where they could be committed to source control and made publicly available. Even values embedded in your compiled application can be extracted, so it's best to externalize them whenever possible.

Virtually every web framework provides a mechanism for loading configuration, and the old .NET Framework version of ASP.NET was no different. It used the `<appsettings>` element in a web.config file to store key-value configuration pairs. At runtime you'd use the static (*wince*) `ConfigurationManager` to load the value for a given key from the file. You could do more advanced things using custom configuration sections, but doing so was painful and so was rarely used, in my experience.

ASP.NET Core gives you a totally revamped experience. At the most basic level, you're still specifying key-value pairs as strings, but instead of getting those values from a single file, now you can load them from multiple sources. You can load values from files, but now they can be in any format you like: JSON, XML, YAML, and so on. Further, you can load values from environment variables, from command-line arguments, from a database, or from a remote service. Or you can create your own custom configuration provider.

DEFINITION ASP.NET Core uses *configuration providers* to load key-value pairs from a variety of sources. Applications can use multiple configuration providers.

The ASP.NET Core configuration model also has the concept of overriding settings. Each configuration provider can define its own settings, or it can overwrite settings from a previous provider. You'll see this incredibly useful feature in action in section 10.2.